Doloso_Felixander_Make_Circle

November 12, 2026

[1]:
```
import tensorflow as tf
print(tf.__version__)

import datetime
print(f"Notebook last run (end-to-end): {datetime.datetime.now()}")
```

```
2.21.0
Notebook last run (end-to-end): 2026-11-12 15:27:14.882955
```

[2]:
```
from sklearn.datasets import make_circles

# Make 1000 examples
n_samples = 1000

# Create circles
X, y = make_circles(n_samples,
                    noise=0.03,
                    random_state=42)
```

[3]:
```
X
```

[3]:
```
array([[ 0.75424625,  0.23148074],
       [-0.75615888,  0.15325888],
       [-0.81539193,  0.17328203],
       …,
       [-0.13690036, -0.81001183],
       [ 0.67036156, -0.76750154],
       [ 0.28105665,  0.96382443]], shape=(1000, 2))
```

[4]:
```
y[:10]
```

[4]:
```
array([1, 1, 1, 1, 0, 1, 1, 1, 1, 0])
```

[5]:
```
# Make dataframe of features and labels
import pandas as pd
circles = pd.DataFrame({"X0":X[:, 0], "X1":X[:, 1], "label":y})
circles.head()
```

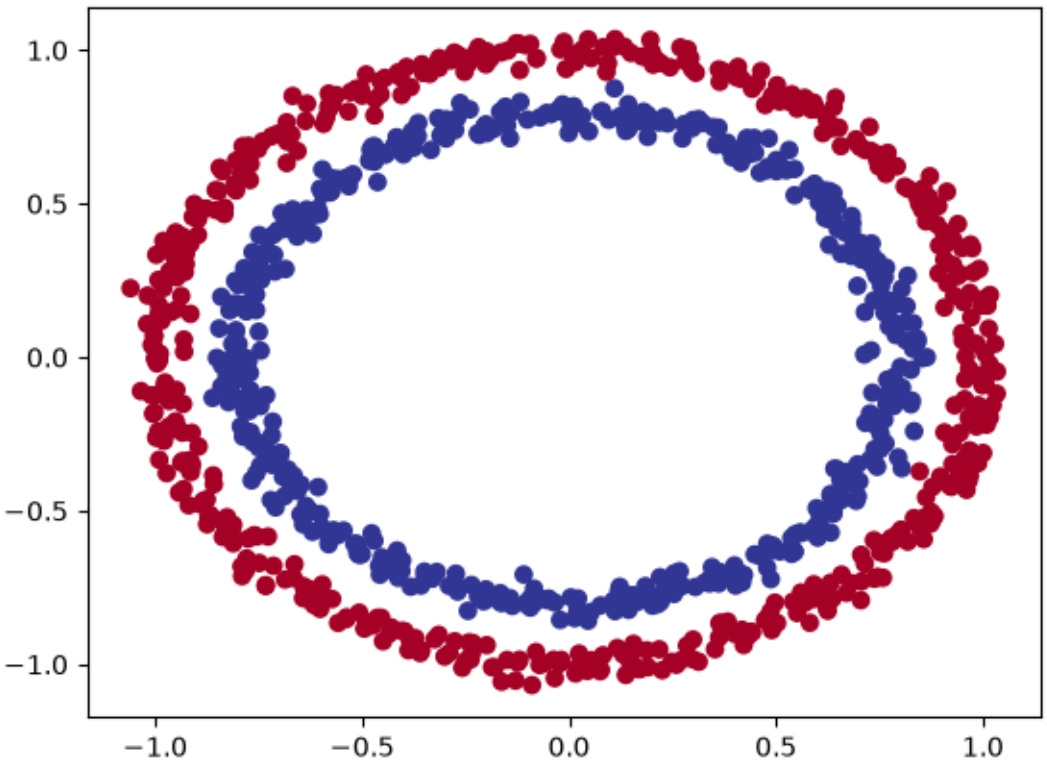
[5]:

	X0	X1	label
0	0.754246	0.231481	1
1	-0.756159	0.153259	1
2	-0.815392	0.173282	1
3	-0.393731	0.692883	1
4	0.442208	-0.896723	0

[6]:
```
# Check out the different labels
circles.label.value_counts()
```

[6]:
```
label
1    500
0    500
Name: count, dtype: int64
```

[7]:
```
# Visualize with a plot
import matplotlib.pyplot as plt
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu);
```



[8]:
```
# Check the shapes of our features and labels
X.shape, y.shape
```

[8]:
```
((1000, 2), (1000,))
```

[9]:
```
# Check how many samples we have
len(X), len(y)
```

[9]:
```
(1000, 1000)
```

[10]:
```
# View the first example of features and labels
X[0], y[0]
```

[10]:
```
(array([0.75424625, 0.23148074]), np.int64(1))
```

[11]:
```
from tensorflow.keras import callbacks

# Set random seed
tf.random.set_seed(42)

# Create a model
model = tf.keras.Sequential([
  tf.keras.layers.Dense(16, activation="relu"), # hidden layer 1, ReLU␣
 ↪activation
  tf.keras.layers.Dense(8, activation="relu"), # hidden layer 2, ReLU activation
  tf.keras.layers.Dense(1, activation="sigmoid") # output layer, sigmoid␣
 ↪activation
])

# Compile the model
model.compile(loss=tf.keras.losses.binary_crossentropy,
                            optimizer=tf.keras.optimizers.Adam(learning_rate=0.
 ↪001),
                            metrics=['accuracy'])

early_stop = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True)

# Fit the model
history = model.fit(X, y,
                        validation_split=0.2,
                        batch_size=32,
                        callbacks=[early_stop],
                        epochs=500,
                        verbose=0)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
Please use WSL2 or the TensorFlow-DirectML plugin.
```

[12]:
```
model.evaluate(X, y)
```

```
32/32             0s 10ms/step -
accuracy: 1.0000 - loss: 5.2262e-04
```

[12]:
```
[0.0005226214416325092, 1.0]
```

[13]:
```
import numpy as np

def plot_decision_boundary(model, X, y):
  """
  Plots the decision boundary created by a model predicting on X.
  This function has been adapted from two phenomenal resources:
   1. CS231n - https://cs231n.github.io/neural-networks-case-study/
   2. Made with ML basics - https://github.com/GokuMohandas/MadeWithML/blob/
 ↪main/notebooks/08_Neural_Networks.ipynb
  """
  # Define the axis boundaries of the plot and create a meshgrid
  x_min, x_max = X[:, 0].min() - 0.1, X[:, 0].max() + 0.1
  y_min, y_max = X[:, 1].min() - 0.1, X[:, 1].max() + 0.1
  xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                       np.linspace(y_min, y_max, 100))

  # Create X values (we're going to predict on all of these)
  x_in = np.c_[xx.ravel(), yy.ravel()] # stack 2D arrays together: https://
 ↪numpy.org/devdocs/reference/generated/numpy.c_.html

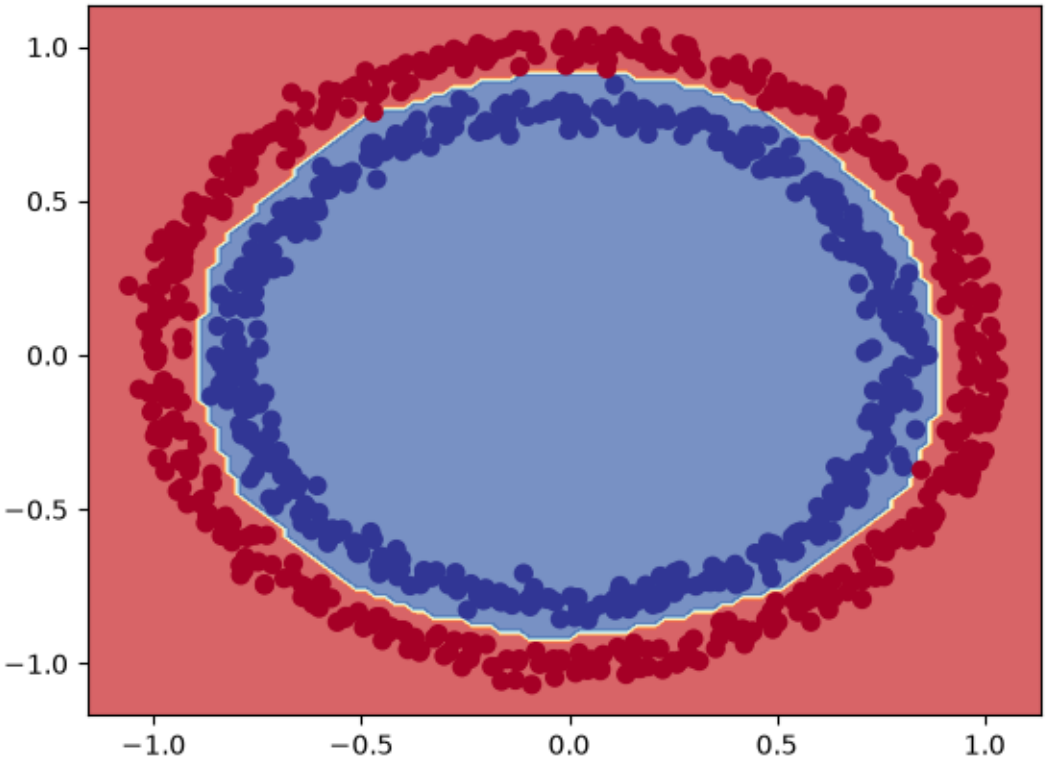
  # Make predictions using the trained model
  y_pred = model.predict(x_in)

  # Check for multi-class
  if model.output_shape[-1] > 1: # checks the final dimension of the model's␣
 ↪output shape, if this is > (greater than) 1, it's multi-class
    print("doing multiclass classification...")
    # We have to reshape our predictions to get them ready for plotting
    y_pred = np.argmax(y_pred, axis=1).reshape(xx.shape)
  else:
    print("doing binary classifcation...")
    y_pred = np.round(np.max(y_pred, axis=1)).reshape(xx.shape)

  # Plot decision boundary
  plt.contourf(xx, yy, y_pred, cmap=plt.cm.RdYlBu, alpha=0.7)
  plt.scatter(X[:, 0], X[:, 1], c=y, s=40, cmap=plt.cm.RdYlBu)
  plt.xlim(xx.min(), xx.max())
  plt.ylim(yy.min(), yy.max())
```

[14]:
```
# View the predictions of the model with relu and sigmoid activations
plot_decision_boundary(model, X, y)
```

```
313/313             1s 4ms/step
doing binary classifcation…
```



[15]:
```
len(X)
```

[15]: 1000

[16]:
```
# Split data into train and test sets
X_train, y_train = X[:800], y[:800] # 80% of the data for the training set
X_test, y_test = X[800:], y[800:] # 20% of the data for the test set

# Check the shapes of the data
X_train.shape, X_test.shape # 800 examples in the training set, 200 examples in␣
 ↪the test set
```

[16]: ((800, 2), (200, 2))

[17]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

[18]:
```
# Set random seed
tf.random.set_seed(42)
# Create the model (same as model_7)
model_1 = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation="relu"), # hidden layer 1, ReLU␣
 ↪activation
    tf.keras.layers.Dense(8, activation="relu"), # hidden layer 2, ReLU␣
 ↪activation
    tf.keras.layers.Dense(1, activation="sigmoid") # output layer, sigmoid␣
 ↪activation
])
# Compile the model
model_1.compile(loss=tf.keras.losses.binary_crossentropy,
                            optimizer=tf.keras.optimizers.Adam(learning_rate=0.
 ↪001),
                            metrics=['accuracy'])

early_stop = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True)

# Fit the model
history = model_1.fit(X_train, y_train,
                        validation_split=0.2,
                        batch_size=32,
                        callbacks=[early_stop],
                        epochs=500,
                        verbose=0)
```

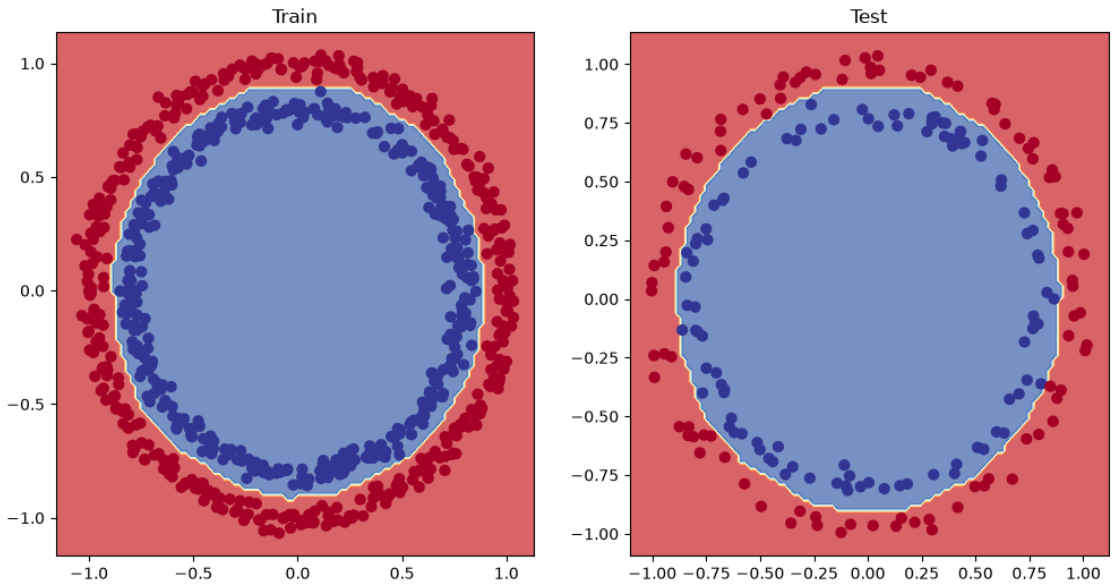
[19]:
```
loss, accuracy = model_1.evaluate(X_test, y_test)
print(f"Model loss on the test set: {loss}")
print(f"Model accuracy on the test set: {100*accuracy:.2f}%")
```

```
7/7             0s 16ms/step -
accuracy: 1.0000 - loss: 0.0047
Model loss on the test set: 0.004746625665575266
Model accuracy on the test set: 100.00%
```

[20]:
```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
```

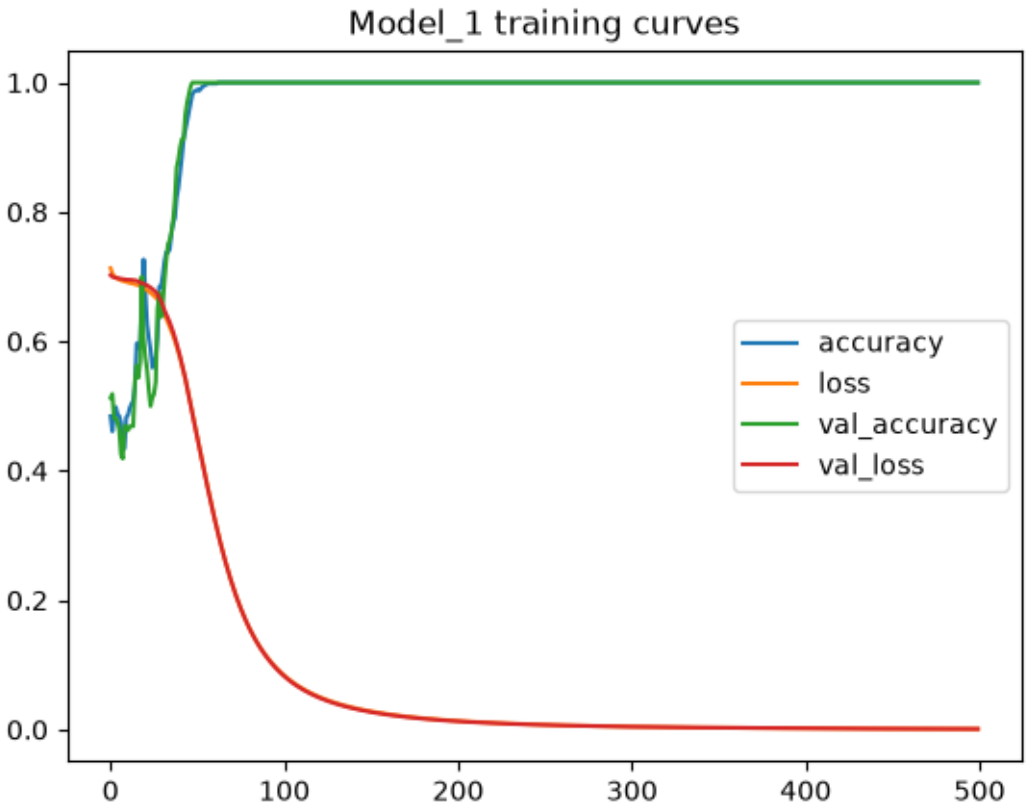
```
plt.title("Train")
plot_decision_boundary(model_1, X=X_train, y=y_train)
plt.subplot(1, 2, 2)
plt.title("Test")
plot_decision_boundary(model_1, X=X_test, y=y_test)
plt.show()
```

```
313/313             1s 3ms/step
doing binary classifcation…
313/313             1s 3ms/step
doing binary classifcation…
```



[21]:
```
pd.DataFrame(history.history).plot()
plt.title("Model_1 training curves")
```

[21]: `Text(0.5, 1.0, 'Model_1 training curves')`


Model_1 training curves
1.0
0.8
0.6
0.4
0.2
0.0
0
100
200
300
400
500
accuracy
loss
val_accuracy
val_loss